

LAPORAN PRAKTIKUM MOBILE PROGRAMMING

MODUL 11

(Manajemen State dengan GetX)



Nama : Dimas Bagus Hari Murti

NIM : 240605110080

Kelas : B

Tanggal : 13 Oktober 2025

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG

GANJIL 2025/2026

I. Tujuan

Percobaan kali ini bertujuan untuk mengeksplorasi teknik State Management modern di Flutter menggunakan library GetX. Fokus utamanya adalah memahami bagaimana cara memisahkan logika bisnis (business logic) dari tampilan antarmuka (UI) agar kode menjadi lebih bersih dan terstruktur. Selain itu, percobaan ini juga bertujuan untuk membandingkan efisiensi antara penggunaan setState() yang bersifat rebuild satu halaman penuh dengan pendekatan reaktif milik GetX yang hanya memperbarui widget yang berubah saja. Diharapkan setelah praktikum ini, saya mampu mengimplementasikan Controller, variabel reaktif (.obs), dan widget pemantau (Obx) untuk membuat aplikasi Tasbih Digital yang responsif secara real-time.

II. Langkah Kerja

- Membuat proyek Flutter baru dengan nama getx_app.
- Menambahkan dependensi pada file pubspec.yaml.
- Membuat struktur folder yang terdiri atas model, view, dan viewmodel.
- Membuat kelas TasbihController yang berisi variabel reaktif counter dan progress, serta method incrementCounter() dan resetCounter().
- Menghubungkan controller dengan tampilan menggunakan widget Obx.
- Menjalankan aplikasi untuk melihat perubahan nilai counter dan progress bar secara realtime.

III. Screenshot Hasil

1. Main.dart

Source code:

```
1 import 'package:flutter/material.dart';
2 import 'package:get/get.dart';
3 import 'view/home.dart';
4
5 void main() {
6   runApp(const MyApp());
7 }
8
9 class MyApp extends StatelessWidget {
10   const MyApp({super.key});
11
12   @override
13   Widget build(BuildContext context) {
14     return GetMaterialApp(
15       debugShowCheckedModeBanner: false,
16       title: 'Tasbih Digital GetX',
17       theme: ThemeData(
18         colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
19         useMaterial3: true,
20       ),
21       home: Home(),
22     );
23   }
24 }
```

File ini adalah gerbang utama aplikasi. Perubahan paling krusial di sini adalah penggantian widget induk dari MaterialApp biasa menjadi GetMaterialApp. Langkah ini sangat penting karena tanpa GetMaterialApp, kita tidak bisa memanfaatkan fitur-fitur canggih GetX seperti manajemen rute tanpa context ataupun penyuntikan dependensi state management yang akan digunakan di halaman-halaman berikutnya.

2. Tasbih_controller.dart

```
1 import 'package:get/get.dart';
2
3 class TasbihController extends GetxController {
4   var counter = 0.0.obs;
5   var progress = 0.0.obs;
6   final double maxCount = 33;
7
8   void incrementCounter() {
9     if (counter < maxCount) {
10       counter.value++;
11       progress.value = (counter.value / maxCount) * 100;
12     }
13   }
14
15   void resetCounter() {
16     counter.value = 0;
17     progress.value = 0;
18   }
19 }
20
```

File ini berperan sebagai "otak" aplikasi yang terpisah dari tampilan. Di sini saya membuat kelas `TasbihController` yang mewarisi `GetXController`. Variabel `counter` dan `progress` diberi akhiran `.obs` (observable), yang artinya variabel ini diawasi perubahannya. Jadi, setiap kali fungsi `incrementCounter` dijalankan dan nilai variabel berubah, `GetX` akan otomatis memberi tahu UI untuk memperbarui tampilan tanpa perlu kita panggil `setState` secara manual. Logika bisnis hitungan dzikir sepenuhnya terjadi di sini, bukan di file UI

3. home.dart

```
import 'package:flutter/material.dart';
import '../viewmodel/tasbih_controller.dart';
import 'package:get/get.dart';

class Home extends StatelessWidget {
  Home({super.key});
  final TasbihController controller = Get.put(TasbihController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color.fromARGB(255, 119, 210, 145),
      body: SafeArea(
        child: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Obx(
                () => Text(
                  '${controller.counter.value.round()}',
                  style: const TextStyle(fontSize: 250),
                ),
              ),
              Obx(
                () => Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 40.0),
                  child: LinearProgressIndicator(
                    value: controller.progress.value / 100,
                    backgroundColor: Colors.white54,
                    color: Colors.amberAccent.shade400,
                    minHeight: 15,
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
              ),
              Obx(
                () => ClipRRect(
                  borderRadius: const BorderRadius.all(Radius.circular(50)),
                  child: InkWell(
                    onTap: controller.incrementCounter,
                    child: Container(
                      decoration: const BoxDecoration(color: Colors.white),
                      padding: const EdgeInsets.all(30),
                      child: const Icon(Icons.fingerprint, size: 100),
                    ),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

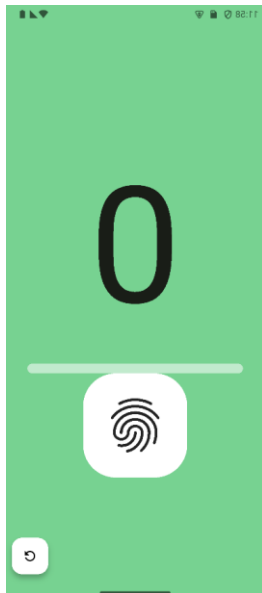
```
), Container
), InkWell
), ClipRRect
),
), Column
), Center
), SafeArea
floatingActionButton: FloatingActionButton(
  onPressed: controller.resetCounter,
  backgroundColor: Colors.white,
  child: const Icon(
    Icons.refresh_outlined,
    color: Colors.black,
  ),
), FloatingActionButton
); Scaffold
}
```

Di halaman Home ini tidak lagi menggunakan `StatefulWidget`, melainkan cukup `StatelessWidget`. Hal ini bisa dilakukan karena state sudah dipegang oleh `TasbihController` yang dipanggil menggunakan `Get.put()`. Untuk menampilkan data yang berubah-ubah (seperti angka hitungan dan progress bar), saya membungkus widget `Text` dan `LinearProgressIndicator` dengan `Obx`. Widget `Obx` ini berfungsi sebagai pendengar (listener); dia hanya akan membangun ulang (rebuild) bagian kecil di dalamnya saja ketika nilai `.obs`

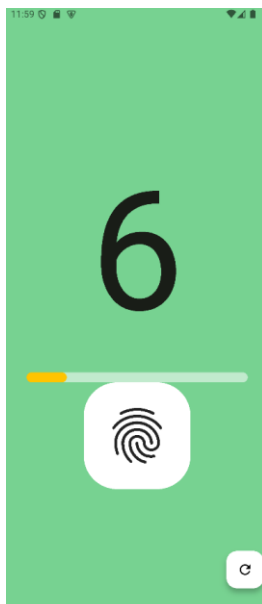
di controller berubah. Ini jauh lebih efisien daripada me-refresh satu halaman penuh.

Output:

Ketika State 0:



Ketika nilai ditambah:



IV. Kesimpulan

Dari percobaan yang telah dilakukan, saya menyimpulkan bahwa penggunaan GetX menawarkan pendekatan yang jauh lebih efisien dan rapi dibandingkan metode konvensional setState. Dengan GetX, kode logika bisnis terisolasi sepenuhnya di dalam Controller, sehingga file UI (view) menjadi lebih bersih dan hanya fokus pada urusan visual. Selain itu, mekanisme reaktif menggunakan variabel .obs dan widget Obx sangat memudahkan pengelolaan data yang dinamis. Aplikasi menjadi lebih responsif karena pembaruan tampilan terjadi secara otomatis dan terarah (hanya widget terkait yang diperbarui), tanpa perlu merender ulang keseluruhan halaman. Hal ini membuktikan bahwa GetX sangat cocok digunakan untuk proyek yang membutuhkan performa tinggi dan struktur kode yang mudah dipelihara (maintainable)